

Podatność SQL Injection może pozwalać na: *

- ☒ nieautoryzowanym dostępem w trybie odczytu lub zapisu do całej bazy danych
- ☒ ominięcie mechanizmu uwierzytelnienia
- ☒ możliwością wykonania kodu w systemie operacyjnym
- ☒ możliwością odczytania wybranych plików.

SQL Injection może (w zależności od uprawnień konta DB i konfiguracji) prowadzić do wszystkich rzeczy z listy — więc tutaj zaznaczyłabym wszystkie 4. Wyjaśnienie po kolei:

- ✓ nieautoryzowany dostęp w trybie odczytu lub zapisu do bazy danych
 - Najczęstszy skutek: napastnik może odczytywać rekordy (np. użytkowników), modyfikować je, usuwać, czasem wykonywać dodatkowe zapytania.
- ✓ ominięcie mechanizmu uwierzytelnienia
 - Klasyk: jeśli logowanie buduje zapytanie SQL z wejścia użytkownika, można doprowadzić do „zalogowania bez hasła” poprzez zmianę warunku w zapytaniu.
- ✓ możliwość wykonania kodu w systemie operacyjnym
 - To zależy od silnika i konfiguracji, ale bywa możliwe np. przez niebezpieczne funkcje/procedury (albo mechanizmy typu uruchamianie poleceń, extended procedures) i odpowiednie uprawnienia DB. Nie zawsze, ale może.
- ✓ możliwość odczytania wybranych plików
 - Również zależy od bazy i uprawnień, ale część DB ma funkcje/komendy do wczytywania plików z serwera bazy (np. do importu), co przy złej konfiguracji może zostać nadużyte.

Jeśli to test jednokrotnego wyboru „co na pewno”, to najpewniejsze i najbardziej uniwersalne byłyby pierwsze dwie (DB read/write + auth bypass). Ale przy sformułowaniu „może pozwalać” poprawne są wszystkie cztery.

Które algorytmy są dostępne w bibliotece OpenSSL do generowania funkcji skrótu? (2 odpowiedzi)

- ☒ MD5
- ☒ SHA-256
- ☐ ROT13
- ☐ Base64
- ☐ XOR

Które algorytmy są dostępne w bibliotece OpenSSL do generowania funkcji skrótu? (2 odpowiedzi)

Poprawne 2 odpowiedzi to:

- ☒ MD5
- ☒ SHA-256

Wyjaśnienie:

- MD5 i SHA-256 to kryptograficzne funkcje skrótu (hash functions) i są dostępne w OpenSSL (np. openssl dgst -md5 i openssl dgst -sha256).
 - ROT13 to prosta zamiana liter (szyfr podstawieniowy), nie hash.
 - Base64 to kodowanie, nie funkcja skrótu (da się odwrócić).
 - XOR to operacja bitowa używana w szyfrowaniu/obfuskacji, ale nie jest funkcją skrótu.
-

Ciąg znaków "aasb763jjab" został zapisany do pliku file.txt za pomocą polecenia *
"echo "aasb763jjab" > file.txt". Aby wygenerować skrót sha-256 dla ciągu
"aasb763jjab" należy wykonać polecenie:

- ☐ echo "aasb763jjab" | sha256sum
- ☐ cat file.txt | sha256sum
- ☒ echo -n "aasb763jjab" | sha256sum
- ☐ sha256sum file.txt
- ☐ sha256sum < file.txt

Ciąg znaków "aasb763jjab" został zapisany do pliku file.txt za pomocą polecenia *
"echo "aasb763jjab" > file.txt". Aby wygenerować skrót sha-256 dla ciągu
"aasb763jjab" należy wykonać polecenie:

```
echo "aasb763jjab" | sha256sum
cat file.txt | sha256sum
☒ echo -n "aasb763jjab" | sha256sum
sha256sum file.txt
sha256sum < file.txt
```

Kluczowy szczegół: polecenie

```
echo "aasb763jjab" > file.txt
```

zapisuje do pliku tekst + znak nowej linii na końcu (bo domyślnie echo dopisuje \n).
Więc w file.txt jest tak naprawdę:

```
aasb763jjab\n
```

Pytanie jednak brzmi: skrót dla ciągu aasb763jjab (bez dopisanego enterem \n).
Dlatego poprawne będzie polecenie, które hashuje dokładnie te znaki bez nowej linii:

```
☒ echo -n "aasb763jjab" | sha256sum
```

Wyjaśnienie pozostałych:

- ☒ echo "aasb763jjab" | sha256sum – tu echo doda \n, więc hash będzie dla aasb763jjab\n
- ☒ cat file.txt | sha256sum – hashuje zawartość pliku, czyli też aasb763jjab\n
- ☒ sha256sum file.txt – j.w., hash pliku z \n
- ☒ sha256sum < file.txt – j.w., hashuje bytes z pliku (z \n)

Jeśli test jest „podchwytliwy” i chcą hash dokładnie tego, co jest w pliku po echo, wtedy poprawne byłyby: `sha256sum file.txt` albo `sha256sum < file.txt` (bo one uwzględnią `\n`).

Ale przy sformułowaniu „dla ciągu `aasb763jjab`” — wybór to `echo -n ... | sha256sum`.

Odbiorca (B) w celu weryfikacji podpisanej przez nadawcę (A) wiadomości: *

- ☐ wykorzysta klucz prywatny A
- ☐ wykorzysta klucz prywatny B
- ☒ wykorzysta klucz publiczny A
- ☐ wykorzysta klucz publiczny B

Odbiorca (B) w celu weryfikacji podpisanej przez nadawcę (A) wiadomości: *

wykorzysta klucz prywatny A

wykorzysta klucz prywatny B

☒ wykorzysta klucz publiczny A

wykorzysta klucz publiczny B

Poprawna odpowiedź to:

☒ „wykorzysta klucz publiczny A”

Wyjaśnienie (krótko i konkretnie)

- Nadawca A podpisuje wiadomość swoim kluczem prywatnym A.
- Odbiorca B weryfikuje podpis używając klucza publicznego A, bo tylko on pasuje do prywatnego A i pozwala sprawdzić, że:
 1. podpis faktycznie zrobił A (autentyczność),
 2. wiadomość nie została zmieniona (integralność).

Dlaczego pozostałe są błędne

- ☒ klucz prywatny A — odbiorca nie powinien go znać
- ☒ klucz prywatny B — nie ma związku z podpisem A
- ☒ klucz publiczny B — też nie pasuje do podpisu A

Jeśli na Twoim teście obok „klucz publiczny A” jest czerwony X, to możliwe, że pytanie dotyczyło szyfrowania dla odbiorcy (tam byłby klucz publiczny B), ale przy sformułowaniu „weryfikacji podpisanej przez A wiadomości” zawsze jest publiczny A.

Które polecenia są poprawne w kontekście tworzenia pliku o rozmiarze 1MB z losową zawartością? (2 odpowiedzi) *

- ☒ `head -c 1048576 /dev/urandom`
- ☒ `dd if=/dev/urandom of=file.txt bs=1M count=1`
- ☐ `cat /dev/random > file.txt`
- ☐ `echo "1MB" > file.txt`
- ☐ `truncate -s 1M file.txt`

Które polecenia są poprawne w kontekście tworzenia pliku o rozmiarze 1MB z losową zawartością? (2 odpowiedzi)

- ☒ `head -c 1048576 /dev/urandom`
 - ☒ `dd if=/dev/urandom of=file.txt bs=1M count=1`
- `cat /dev/random > file.txt`
`echo "1MB"> file.txt`
`truncate -s 1M file.txt`

Dwie sensowne komendy do utworzenia pliku 1 MiB z losową zawartością to:

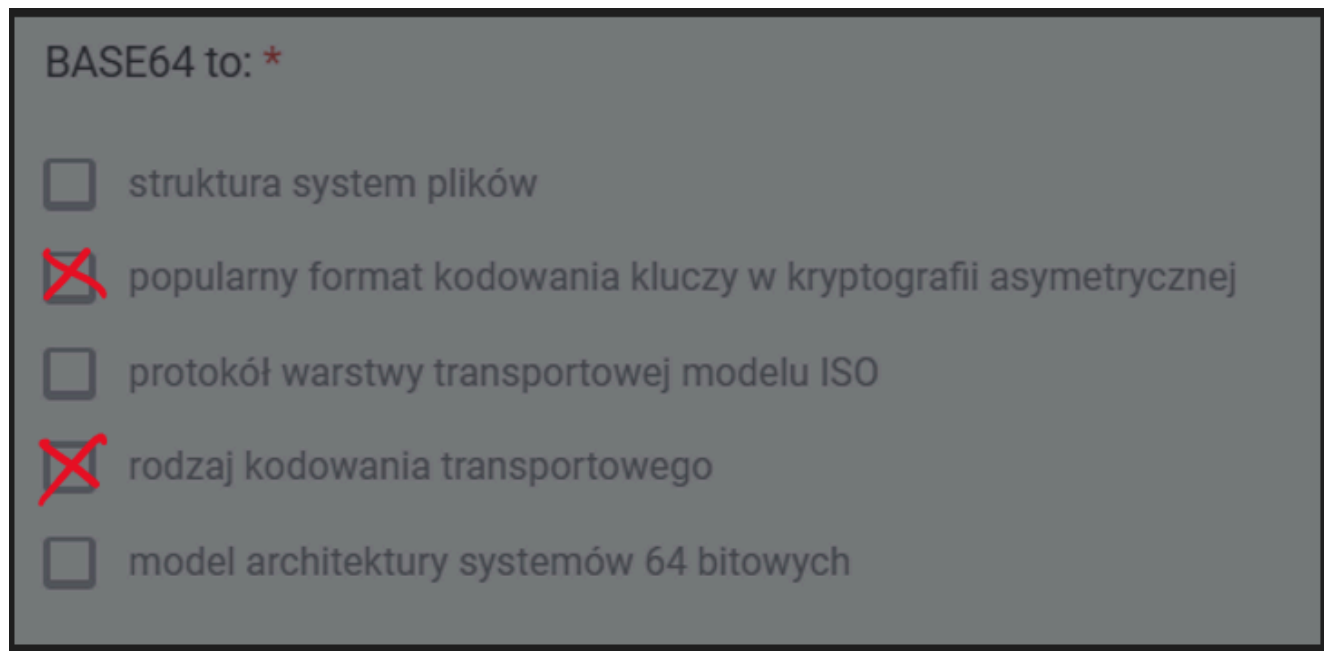
- ☒ `dd if=/dev/urandom of=file.txt bs=1M count=1`
 - `if=/dev/urandom` daje losowe bajty
 - `bs=1M count=1` zapisuje dokładnie 1 MiB do `file.txt`
- ☒ `head -c 1048576 /dev/urandom > file.txt` (uwaga: potrzebne jest przekierowanie do pliku)

- `-c 1048576` pobiera dokładnie 1 048 576 bajtów (= 1 MiB)
- `file.txt` zapisuje wynik do pliku
Bez `> file.txt` ta komenda tylko wypisze dane na ekran i nie utworzy pliku.

Pozostałe opcje są niepoprawne w tym kontekście, bo:

- `cat /dev/random > file.txt` – nie kończy się po 1 MiB i może się blokować (zbiera „prawdziwą” entropię)
- `echo "1MB" > file.txt` – zapisze tekst „1MB”, nie 1 MiB danych i nie losowe
- `truncate -s 1M file.txt` – ustawi rozmiar 1 MiB, ale zawartość nie będzie losowa (zwykle zera / plik rzadki)

Jeśli w Twoim teście pierwsza opcja jest dosłownie bez > file.txt, to formalnie nie spełnia „tworzenia pliku” — ale w praktyce to właśnie ona jest drugim poprawnym sposobem po dodaniu przekierowania.



BASE64 to:

struktura system plików

✓ popularny format kodowania kluczy w kryptografii asymetrycznej

protokół warstwy transportowej modelu ISO

✓ rodzaj kodowania transportowego

model architektury systemów 64 bitowych

Base64 to kodowanie binarnych danych na znaki tekstowe (A–Z, a–z, 0–9, +, / oraz ewentualnie = jako padding). Nie jest to szyfrowanie ani kompresja — tylko sposób zapisu/przesyłu binarnych bajtów w formie tekstu.

✓ 1) „rodzaj kodowania transportowego” — TAK

- Base64 powstało właśnie po to, żeby dało się przenosić dane binarne (np. obrazek, klucz, plik) w systemach, które oczekują tekstu (np. e-mail, JSON, XML).
- Przykład: załączniki w e-mailach (MIME), dane w JSON Web Token (często base64url), itp.

✓ 2) „popularny format kodowania kluczy w kryptografii asymetrycznej” — TAK

- Klucze i certyfikaty często są w formatach, które zawierają Base64.
- Najprostszy przykład: PEM (-----BEGIN CERTIFICATE----- ...) — środek to Base64 danych DER.
- To nie znaczy, że Base64 jest kryptografią — tylko, że klucze/certyfikaty są często tak zapisywane w plikach tekstowych.

✗ 3) „struktura systemu plików” — NIE

- To by dotyczyło rzeczy typu ext4, NTFS, FAT itp.
- Base64 nie ma nic wspólnego z organizacją danych na dysku.

✗ 4) „protokół warstwy transportowej modelu ISO” — NIE

- Warstwa transportowa to np. TCP/UDP (w praktyce modelu TCP/IP).
- Base64 nie jest protokołem sieciowym — to tylko kodowanie danych.

✗ 5) „model architektury systemów 64 bitowych” — NIE

- „64” w Base64 odnosi się do liczby znaków w alfabecie (64 symbole), a nie do 64-bitowego procesora.

Na podstawie poniższego obrazka można powiedzieć, że szczegółowe informacje o procesie xfce4-session można znaleźć w katalogu:

The screenshot shows a terminal window with a list of processes. The process 'xfce4-session' is highlighted. Below the terminal, there is a file browser interface showing a list of directories. The directory '/proc/828/' is selected, indicated by a green checkmark. A dialog box is overlaid on the file browser, asking the user to log in to continue.

PID	TTY	STAT	TIME	COMMAND
797	?	Ss	0:00	/lib/systemd/systemd --user
798	?	S	0:00	(sd-pam)
817	?	S<sl	0:00	/usr/bin/pipewire
818	?	S<sl	0:00	/usr/bin/pipewire-media-session
819	?	S<sl	0:00	/usr/bin/pulseaudio --daemonize=no --log-target=jou
821	?	Ss	0:00	/usr/bin/dbus-daemon --session --address=systemd: -
824	?	Sl	0:00	/usr/bin/gnome-keyring-daemon --daemonize --login
828	?	Ssl	0:00	xfce4-session
876	?	Ss	0:00	/usr/bin/ssh-agent x-session-manager

Na podstawie poniższego obrazka można powiedzieć, że szczegółowe informacje o procesie xfce4-session można znaleźć w katalogu:

- ☐ /dev/proc/828
- ☒ /proc/828/
- ☐ /proc/xfce4-session
- ☐ /opt/828/xfce4-session
- ☐ /var/log/xfce4-session
- ☐ /var/log/828

Zaloguj się, by kontynuować

Aby wypełnić ten formularz, musisz się zalogować.

[Zgłoś nadużycie](#)

ZALOGUJ SIĘ

obrazek z terminalem PID | TTY | STAT | TIME | COMAND

/dev/proc/828
✓ /proc/828/
/proc/xfce4-session
/opt/828/xfce4-session
/var/log/xfce4-session
/var/log/828

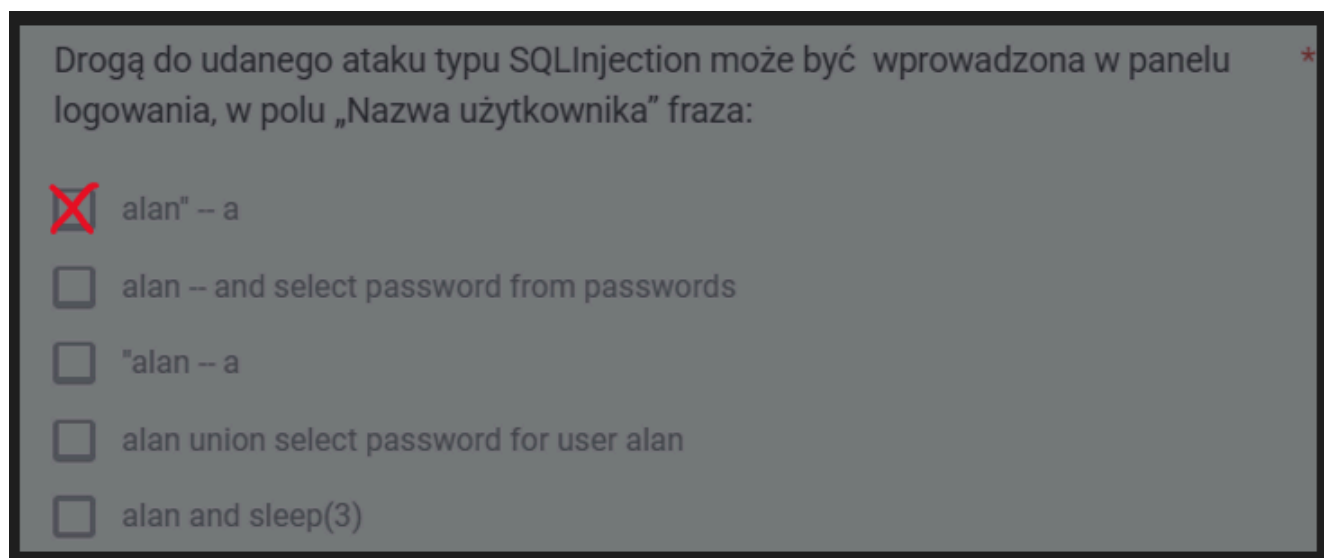
Poprawna odpowiedź to:

✓ /proc/828/

Dlaczego: na obrazku widać listę procesów i że xfce4-session ma PID = 828. W Linuksie szczegółowe informacje o procesie (m.in. cmdline, environ, cwd, exe, fd/, status, maps) są w pseudo-systemie plików /proc//, czyli tutaj /proc/828/.

Pozostałe propozycje są błędne, bo:

- /dev/proc/828 – taka standardowa ścieżka nie istnieje
- /proc/xfce4-session – /proc jest indeksowane PID-ami, nie nazwami procesów
- /opt/... i /var/log/... – to nie jest miejsce na metadane procesu (logi mogą być w /var/log, ale nie "szczegółowe informacje o procesie" w sensie /proc).



Drogą do udanego ataku typu SQLInjection może być wprowadzona w panelu logowania, w polu „Nazwa użytkownika” fraza:

✓ alan" -- a

alan -- and select password from passwords

"alan -- a

alan union select password for user alan

alan and sleep(3)

Żeby wstrzyknięcie z pola tekstowego zadziało w typowym (podatnym) logowaniu, zwykle muszą zajść naraz rzeczy typu:

1. Wyjście z literału tekstowego (domknięcie cudzysłowu, którym aplikacja otacza input – najczęściej ', rzadziej ").
2. Dodanie jakiegoś operatora/składni SQL poza stringiem.
3. Często też zneutralizowanie reszty zapytania komentarzem (-- ..., / ... /), bo aplikacja dopisuje np. dalsze warunki.

Jeśli nie ma kroku (1), to reszta (--, union, select, sleep) jest zwykle tylko częścią tekstu username i nie „wykonuje się”.

1. alan" -- a

Co to próbuje zrobić: zakończyć string przez " i „uciąć resztę” komentarzem -- a.

Czy ma szansę zadziałać?

- Tylko wtedy, gdy aplikacja naprawdę używa " jako ogranicznika stringów w tym miejscu (np. skleja ... username="WEJŚCIE" ...).
- W wielu DB (PostgreSQL, standard SQL) " jest raczej do identyfikatorów, a stringi są w '...', więc w takich warunkach " nic nie „zamyka” i nie ma efektu.
- Plus: -- a ma spację po --, a to jest ważne w części silników/trybów.

Egzaminowa konkluzja: to jest jedyna odpowiedź z listy, która w ogóle wygląda jak klasyczny schemat „zamknij + skomentuj”, ale jest mocno zależna od tego, czy delimiterem jest ".

2. alan -- and select password from passwords

Co to próbuje zrobić: użyć -- i potem coś dopisać.

Dlaczego najczęściej nie działa:

- Jeśli input jest w '...' albo "...", to -- jest wewnątrz stringa, więc nie zaczyna komentarza — to po prostu znaki w nazwie użytkownika.
- Nawet gdyby było poza stringiem, fragment and select ... nie jest poprawną składnią w tym miejscu w typowym zapytaniu (AND oczekuje warunku logicznego, a nie gołego SELECT).

Egzaminowa konkluzja: wygląda „hakująco”, ale składniowo jest podejrzane i zwykle pozostaje zwykłym tekstem.

3. "alan -- a

Co to próbuje zrobić: też wejść w grę z " i komentarzem.

Dlaczego to jest problematyczne składniowo:

- Zaczynanie od " jest często gorsze niż kończenie nim. W typowym sklejeniu typu username=" + input + " możesz dostać coś w stylu username=""alan ... co daje dziwne zlepienie tokenów (""alan) i często kończy się błędem składni.
- Znowu: zależność od tego, czy " jest w ogóle delimitrem stringa.

Egzaminowa konkluzja: mniej sensowne niż (1), bo łatwiej o błąd składni już na starcie.

4. alan union select password for user alan

Co to próbuje zrobić: atak typu UNION SELECT.

Dlaczego to zwykle nie pasuje do pola "username":

- UNION SELECT działa tylko, gdy wstrzykujesz go w kontekst zapytania SELECT, w miejscu gdzie parser oczekuje dalszej części SELECT-a, i gdy zgadza się liczba/typy kolumn.
- W polu username, jeśli nie domkniesz stringa, to union select ... jest po prostu tekstem.
- Nawet jeśli by było poza stringiem, ten konkretny fragment jest składniowo nienaturalny: for user alan nie jest standardowym SQL (spodziewałbyś się raczej FROM ... WHERE ... itp.).

Egzaminowa konkluzja: wygląda jak „buzzword” z SQLi, ale w tej postaci i w tym miejscu jest mało wiarygodne.

5. alan and sleep(3)

Co to próbuje zrobić: time-based (opóźnienie).

Dlaczego zwykle nie działa / jest niepewne:

- Bez wyjścia z cudzysłowu to tylko tekst.
- Funkcja sleep jest silnie zależna od bazy: w MySQL jest SLEEP(3), w PostgreSQL raczej pg_sleep(3) itd.
- Nawet w MySQL SLEEP(3) zwraca wartość liczbową i użycie jej w AND ... wymaga, by całość miała sens logiczny — co w praktyce często kończy się albo brakiem efektu, albo błędem, albo zmianą logiki w nieprzewidywalny sposób.

Egzaminowa konkluzja: koncept time-based jest prawdziwy, ale ta odpowiedź jest słaba, bo nie rozwiązuje problemu „wyjścia ze stringa” i jest zależna od DB.

W kryptosystemie RSA: *

- ☐ odbierane wiadomości deszyfruje się kluczami publicznymi nadawcy
- ☐ odbierane wiadomości deszyfruje się kluczami publicznymi odbiorcy
- ☒ odbierane wiadomości deszyfruje się kluczami prywatnymi odbiorcy
- ☐ odbierane wiadomości deszyfruje się kluczami prywatnymi nadawcy

W kryptosystemie RSA:

odbierane wiadomości deszyfruje się kluczami publicznymi nadawcy

odbierane wiadomości deszyfruje się kluczami publicznymi odbiorcy

☒ odbierane wiadomości deszyfruje się kluczami prywatnymi odbiorcy

odbierane wiadomości deszyfruje się kluczami prywatnymi nadawcy

W RSA, gdy mówimy o szyfrowaniu dla poufności (najczęstszy sens pytania „odszyfrowuje się wiadomości”), to:

☒ odbierane wiadomości deszyfruje się kluczami prywatnymi odbiorcy

Dlaczego

- Nadawca szyfruje wiadomość kluczem publicznym odbiorcy (każdy może go znać).
- Tylko odbiorca ma pasujący klucz prywatny, więc tylko on może odszyfrować.

Czemu pozostałe są błędne

- ☒ klucze publiczne (nadawcy lub odbiorcy) nie służą do standardowego odszyfrowania w scenariuszu poufności.
- ☒ klucz prywatny nadawcy jest używany do podpisu, a nie do odszyfrowywania wiadomości zaszyfrowanych dla odbiorcy.

(Uwaga: w RSA „odwrócone” użycie kluczy pojawia się w kontekście podpisów: podpis tworzy się prywatnym nadawcy, a weryfikuje publicznym nadawcy — ale to nie jest „odszyfrowywanie odebranych wiadomości” w sensie poufności.)

OpenSSL to: *

- ☐ otwarty protokół komunikacyjny Secure Socket Layer
- ☒ zestaw narzędzi dla protokołów TLS i SSL
- ☐ standard kryptograficzny dla protokołu HTTPS i S-HTTP
- ☒ biblioteka kryptograficzna ogólnego przeznaczenia
- ☐ Memorandum dotyczące podstaw bezpieczeństwa komunikacyjnego

OpenSSL to:

otwarty protokół komunikacyjny Secure Socket Layer

✓ zestaw narzędzi dla protokołów TLS i SSL

standard kryptograficzny dla protokołu HTTPS i S-HTTP

✓ biblioteka kryptograficzna ogólnego przeznaczenia

Memorandum dotyczące podstaw bezpieczeństwa komunikacyjnego

Poprawne są te dwie odpowiedzi:

✓ zestaw narzędzi dla protokołów TLS i SSL

✓ biblioteka kryptograficzna ogólnego przeznaczenia

Wyjaśnienie

- OpenSSL to projekt, który dostarcza:
- bibliotekę (libssl + libcrypto) do kryptografii i TLS/SSL,
- oraz narzędzie wiersza poleceń openssl do operacji typu generowanie kluczy, certyfikatów, testy połączeń TLS itd.

Dlaczego pozostałe są błędne

- ✗ „otwarty protokół komunikacyjny Secure Socket Layer” – OpenSSL nie jest protokołem, tylko implementacją/narzędziami.
- ✗ „standard kryptograficzny dla HTTPS i S-HTTP” – OpenSSL nie jest standardem; HTTPS używa TLS, a OpenSSL to jedna z implementacji.
- ✗ „Memorandum ...” – to nie ten typ rzeczy (brzmi jak dokument/wytyczne, a nie biblioteka).

Poniżej przedsta

Zaloguj się, by kontynuować

Aby wypełnić ten formularz, musisz się zalogować.

[Zgłoś nadużycie](#)

ZALOGUJ SIĘ

```
-----BEGIN PUBLIC KEY-----
MIICIjANBgkqhkiG9w0BAQEuAQkAPQ==
Sqb4yFJHSqNT
kw3Ya18U12nz
2GrjfDe64Ncl
waGGr3fPeBT0
KwFzdNJW8Ym7
ApkiLSCAQiJZ
RDQ087p8UUFKped1f4tG2330020AyDQ0CLBN0jw5w+xe1endn0GhYvzC61uIybly
90tLBrDerGdzw/s7QsoPkmt4mPBHHuWTwdGMwXl+9p91uLLgbgL8kztBWsLlgKG9
q3ADkmHxxFEwxvcXQMLXNfyU53g0qAQw3zEAfyTa+NRm9WaXIcW8p41T+BvWsfFQ
ExvpzNf0d9dQYLRNZzB9ZMdSdgP1k11jNV2viBUyDTQmy0d3aHHQMZZQMEVc9fnL
hRH7KvvCu1kL0hPuoYOCWAMCAwEAAQ==
-----END PUBLIC KEY-----
```

☒ klucz publiczny w formacie pem

☐ klucz publiczny w formacie binarnym

☒ jeden z pary kluczy zapisany z wykorzystaniem kodowania base64

☐ klucz publiczny w postaci heksadecymalnej

☐ klucz prywatny w formacie base64

Poniżej przedsta

- ☒ klucz publiczny w formacie pem
- klucz publiczny w formacie binarnym
- ☒ jeden z pary kluczy zapisany z wykorzystaniem kodowania base64
- klucz publiczny w postaci heksadecymalnej
- klucz prywatny w formacie base64

Na obrazku widać nagłówki:

-----BEGIN PUBLIC KEY----- ... -----END PUBLIC KEY-----

To jest klucz publiczny w formacie PEM – czyli tekstowa „koperta”, a w środku znajduje się Base64 zakodowana binarna struktura (DER).

Zaznacz:

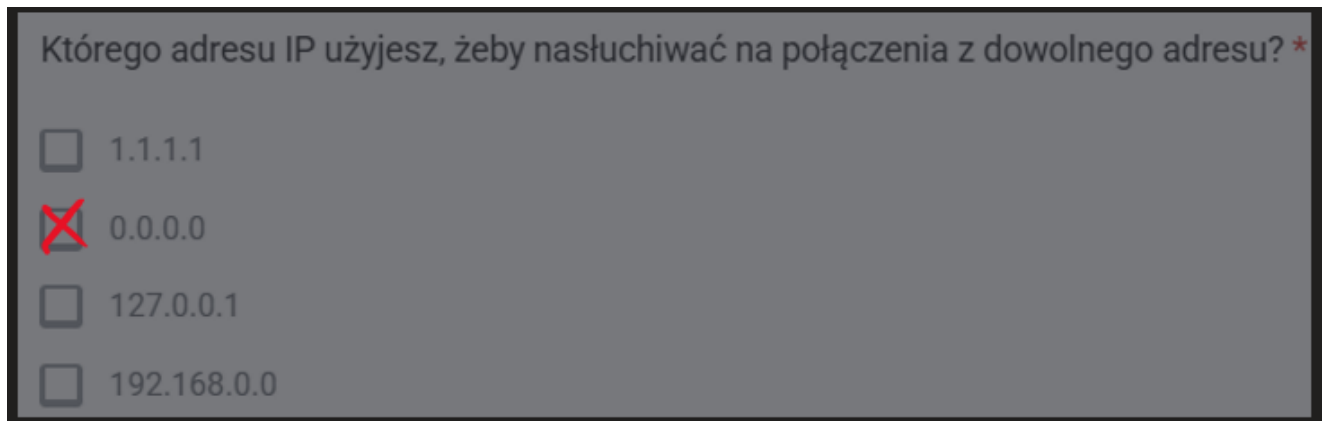
- ☒ klucz publiczny w formacie pem
- ☒ jeden z pary kluczy zapisany z wykorzystaniem kodowania base64

Dlaczego pozostałe nie:

✗ klucz publiczny w formacie binarnym – nie, bo widzisz czytelny tekst z nagłówkami (binarny byłby „krzaczki”/nieczytelne bajty)

✗ klucz publiczny w postaci heksadecymalnej – hex wyglądałby jak długi ciąg znaków 0–9 i a–f, a tu jest Base64 (litery, cyfry, +, /, =)

✗ klucz prywatny w formacie base64 – nagłówek mówi wyraźnie PUBLIC KEY, a prywatny miałby zwykle BEGIN PRIVATE KEY albo BEGIN RSA PRIVATE KEY



Którego adresu IP użyjesz, żeby nasłuchiwać na połączenia z dowolnego adresu? *

☐ 1.1.1.1

☒ 0.0.0.0

☐ 127.0.0.1

☐ 192.168.0.0

Którego adresu IP użyjesz, żeby nasłuchiwać na połączenia z dowolnego adresu? *

1.1.1.1

✓ 0.0.0.0

127.0.0.1

192.168.0.0

✓ 0.0.0.0

Wyjaśnienie: 0.0.0.0 jako adres bind/listen oznacza „nasłuchuj na wszystkich interfejsach” (INADDR_ANY), czyli przyjmuj połączenia z dowolnego adresu, jaki dochodzi do hosta.

Pozostałe:

- 127.0.0.1 → tylko lokalnie (loopback), z zewnątrz się nie połączysz
 - 1.1.1.1 → to konkretny publiczny adres (Cloudflare DNS), nie „wszystkie”
 - 192.168.0.0 → to adres sieci, nie adres hosta do nasłuchu
-

Poniższy ciąg znaków może zostać wykorzystany w ataku o nazwie: *

```

```

- ☒ Cross Site Scripting
- ☐ SQL Injection
- ☐ Reflected JavaScript Attack
- ☐ Improper HTML Data Validation

Poniższy ciąg znaków może zostać wykorzystany w ataku o nazwie:

```

```

- ☒ Cross Site Scripting
- ☐ SQL Injection
- ☐ Reflected JavaScript Attack
- ☐ Improper HTML Data Validation

To jest przykład XSS (Cross-Site Scripting).

Dlaczego: wstrzykujesz znacznik HTML

z atrybutem zdarzenia onerror, który uruchamia JavaScript (tu: alert(...)) w kontekście strony, jeśli obrazek się nie załaduje. To jest klasyczny mechanizm XSS (często reflected albo stored — zależy gdzie ten ciąg trafia).

Z podanych opcji zaznacz:

- ☒ Cross Site Scripting

Pozostałe:

- ☒ SQL Injection — to dotyczy zapytań do bazy, a nie HTML/JS.
 - ☒ Reflected JavaScript Attack — to nie jest standardowa nazwa klasy podatności; jeśli już, to Reflected XSS.
 - ☒ Improper HTML Data Validation — to raczej opis przyczyny (zła walidacja/escape), a nie nazwa ataku. Jeśli pytanie wymaga „nazwa ataku”, to XSS.
-

Które z poniższych adresów IP są zarezerwowane do celów specjalnych? (3 odpowiedzi)

☒ 127.0.0.1

☒ 0.0.0.0

☐ 192.168.1.1

☒ 10.0.0.0

☐ 8.8.8.8

Które z poniższych adresów IP są zarezerwowane do celów specjalnych? (3 odpowiedzi)

☒ 127.0.0.1

☒ 0.0.0.0

192.168.1.1

☒ 10.0.0.0

8.8.8.8

Trzy adresy „do celów specjalnych” z tej listy to:

☒ 127.0.0.1 – loopback (localhost)

☒ 0.0.0.0 – adres nieokreślony / „this host, this network” / INADDR_ANY (specjalne znaczenie)

☒ 10.0.0.0 – początek prywatnej puli 10.0.0.0/8 (RFC1918)

Pozostałe:

- ☒ 192.168.1.1 – też jest prywatny adres, ale pytanie zwykle oczekuje wskazania zarezerwowanych zakresów, a tu podany jest pojedynczy host z puli 192.168.0.0/16 (nie „specjalny” jako pojedynczy adres).
 - ☒ 8.8.8.8 – publiczny DNS Google, normalny publiczny adres.
-

Która z grup kodów odpowiedzi HTTP oznacza, że coś poszło nie tak po stronie serwera? *

☐ 200

☐ 300

☐ 400

☒ 500

Która z grup kodów odpowiedzi HTTP oznacza, że coś poszło nie tak po stronie serwera?

200

300

400

☒ 500

☒ 500 (czyli kody z grupy 5xx)

Wyjaśnienie:

- 2xx – sukces (np. 200 OK)
 - 3xx – przekierowania
 - 4xx – błąd po stronie klienta (np. 404, 401)
 - 5xx – błąd po stronie serwera (np. 500 Internal Server Error, 502, 503)
-

można powiedzieć, że:

```
(kali@kali)-[~/Downloads]
$ openssl x509 -in www-debian-org(1).pem -text
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      91:2b:08:4a:cf:0c:18:a7:53:f6:d6:2e:25:a7:5f:5a
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: C = US, O = Internet Security Research Group, CN = ISRG Root X1
    Validity
      Not Before: Sep  4 00:00:00 2020 GMT
      Not After : Sep 15 16:00:00 2025 GMT
    Subject: C = US, O = Let's Encrypt, CN = R3
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      Public-Key: (2048 bit)
      Modulus:
        00:bb:02:15:28:cc:f6:a0:94:d3:0f:12:ec:8d:55:
        92:c3:f8:82:f1:99:a6:7a:42:88:a7:5d:26:aa:b5:
        2b:b9:c5:4c:b1:af:8e:6b:f9:75:c8:a3:d7:0f:47:
```

- ☐ wystawcą certyfikatu jest "Let's Encrypt"
- ☒ wystawcą certyfikatu jest "Internet Security Research Group"
- ☐ wystawca certyfikatu jest "[debian.org](https://www.debian.org)"
- ☒ klucz publiczny "Let's Encrypt" ma rozmiar 2048 bitów
- ☐ Klucz publiczny "Internet Security Research Group" ma rozmiar 2048 bitów

można powiedzieć, że:

```
(kali@kali)-[~/Downloads]
openssl x509 -in www-debian-org(1).pem -text
Certificate:
Data:
Version: 3 (0-2)
Serial Number:
91:2b:08:43:cf:0c:18:a7:53:f6:d6:2e:25:a7:5f:5a
Signature Algorithm: sha256WithRSAEncryption
Issuer: C=US, O= Internet Security Research Group, CN= ISRG Root X1 Validity
Not Before: Sep 4 00:00:00 2020 GMT
Not After: Sep 15 16:00:00 2025 GMT
Subject: C=US, O= Let's Encrypt, CN = R3
Subject Public Key Info:
Public Key Algorithm: rsaEncryption Public-Key: (2048 bit)
Modulus:
00:bb:02:15:28: cc:f6:a0:94:d3:0f:12:ec:8d:55:
92:c3:f8:82:f1:99:a6:7a:42:88:a7:5d:26:aa:b5:
2b:b9:c5:4c:b1:af:8e:6b:19:75:c8:a3:d7:0f:47:
```

wystawcą certyfikatu jest "Let's Encrypt"

✓ wystawcą certyfikatu jest "Internet Security Research Group"

wystawca certyfikatu jest "debian.org"

✓ klucz publiczny "Let's Encrypt" ma rozmiar 2048 bitów

Klucz publiczny "Internet Security Research Group" ma rozmiar 2048 bitów

Z tego, co widać na zrzucie z openssl x509 -in ... -text, da się odczytać dwie kluczowe linie:

- Issuer: O = Internet Security Research Group, CN = ISRG Root X1
- Subject: O = Let's Encrypt, CN = R3
- oraz Public-Key: (2048 bit) w sekcji Subject Public Key Info (czyli klucz podmiotu/Subject, nie wystawcy).

Co z tego wynika (poprawne odpowiedzi)

✓ „wystawcą certyfikatu jest 'Internet Security Research Group'”

Bo Issuer to wystawca.

✓ „klucz publiczny 'Let's Encrypt' ma rozmiar 2048 bitów”

Bo Subject to Let's Encrypt (R3), a rozmiar klucza publicznego pokazany w certyfikacie dotyczy właśnie Subject (widać (2048 bit)).

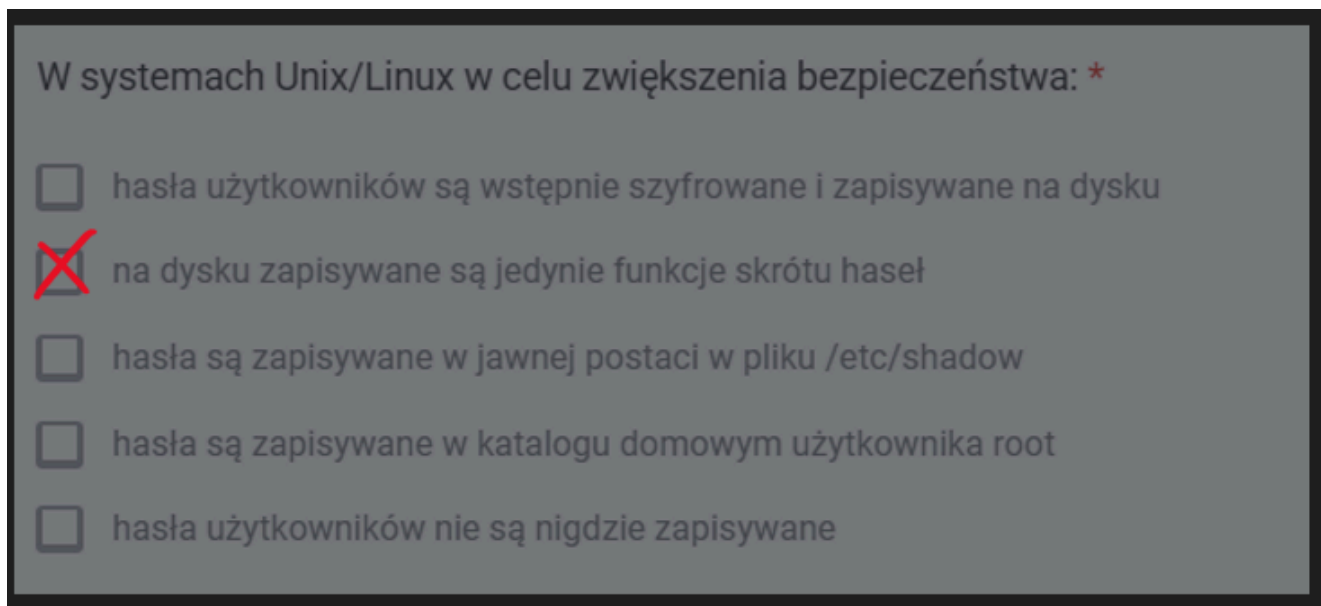
Co jest błędne / nie wynika z obrazka

✗ „wystawcą certyfikatu jest 'Let's Encrypt'" – Let's Encrypt jest tu Subject, a nie Issuer.

✗ „wystawcą certyfikatu jest 'debian.org'" – nigdzie tego nie ma w Issuer/Subject.

✗ „Klucz publiczny 'Internet Security Research Group' ma rozmiar 2048 bitów" – na tym wydruku widzisz klucz Subject, nie klucz wystawcy. Z samego tego certyfikatu nie wyciągniesz rozmiaru klucza Issuer (to byłby w certyfikacie wystawcy/roota).

Jeśli chcesz, mogę Ci też powiedzieć proste „hasło" na test: Issuer = wystawca, Subject = dla kogo jest certyfikat.



W systemach Unix/Linux w celu zwiększenia bezpieczeństwa:

hasła użytkowników są wstępnie szyfrowane i zapisywane na dysku

✓ na dysku zapisywane są jedynie funkcje skrótu haseł

hasła są zapisywane w jawnej postaci w pliku /etc/shadow

hasła są zapisywane w katalogu domowym użytkownika root

hasła użytkowników nie są nigdzie zapisywane

Poprawna odpowiedź to:

✓ „na dysku zapisywane są jedynie funkcje skrótu haseł"

Wyjaśnienie

W Unix/Linux hasła nie są przechowywane w jawnej postaci. System zapisuje w /etc/shadow (zwykle) hash hasła wraz z parametrami (np. solą i informacją o algorytmie). Przy logowaniu system hashuje podane hasło i porównuje z tym, co jest zapisane.

Czemu pozostałe są błędne

- ❌ „hasła są wstępnie szyfrowane i zapisywane” — to nie szyfrowanie (odwracalne), tylko hashowanie (jednokierunkowe).
 - ❌ „hasła są w jawnej postaci w /etc/shadow” — nie, są tam hashe.
 - ❌ „w katalogu domowym roota” — nie.
 - ❌ „hasła nie są nigdzie zapisywane” — są (w postaci hashy) właśnie po to, by dało się je weryfikować.
-

Które tryby szyfrowania są dostępne dla algorytmu AES? (3 odpowiedzi) *

- ☒ ECB
- ☒ CBC
- ☒ CFB
- ☐ ROT
- ☐ BASE

Które tryby szyfrowania są dostępne dla algorytmu AES? (3 odpowiedzi)

- ☒ ECB
- ☒ CBC
- ☒ CFB
- ROT
- BASE

Zaznacz te 3 tryby AES:

- ☒ ECB
- ☒ CBC
- ☒ CFB

Wyjaśnienie:

- ECB (Electronic Codebook), CBC (Cipher Block Chaining) i CFB (Cipher Feedback) to klasyczne tryby pracy szyfru blokowego (AES jest szyfrem blokowym).
 - ROT to nie tryb AES, tylko proste szyfry przesuwne (np. ROT13).
 - BASE nie jest trybem szyfrowania (kojarzy się raczej z Base64, czyli kodowaniem).
-

Poniżej przedstawiono fragment podręcznika systemowego dla polecenia openssl. *
Szyfrowanie pliku data.txt algorytmem aes w trybie wiązania zaszyfrowanych bloków, z blokami o rozmiarze 256 bitów z hasłem "as34Dw9P2IsH" przedstawia polecenie:

```
openssl enc -ciphername [-AadePp] [-base64] [-bufsize number] [-debug]
[-in file] [-iv IV] [-K key] [-k password] [-kfile file]
[-md digest] [-none] [-nopad] [-nosalt] [-out file] [-pass arg]
[-S salt] [-salt]
```

- ☒ openssl enc -e -aes-256-cbc -in data.txt -out data.enc -k as34Dw9P2IsH
- ☐ openssl enc -e -aes-256-ebc -in data.txt -out data.enc -k as34Dw9P2IsH
- ☐ openssl enc -d -aes-256-cbc -in data.txt -out data.enc -K as34Dw9P2IsH
- ☐ openssl enc

Zaloguj się, by kontynuować

Poniżej przedstawiono fragment podręcznika systemowego dla polecenia openssl. *
Szyfrowanie pliku data.txt algorytmem aes w trybie wiązania zaszyfrowanych bloków, z blokami o rozmiarze 256 bitów z hasłem "as34Dw9P2ISH" przedstawia polecenie:

```
openssl enc -ciphername [-AadePp] [-base64] [-bufsize_number] [-debug] [-in file]
[-iv IV] [-K key] [-k password] [-kfile file]
[-md digest] [-none] [-nopad] [-nosalt] [-out file] [-pass arg] [-S salt] [-salt]
```

```
✓ openssl enc -e -aes-256-cbc -in data.txt -out data.enc -k as34Dw9P2IsH
openssl enc -e -aes-256-ebc -in data.txt -out data.enc -k as34Dw9P2IsH
openssl enc -d -aes-256-cbc -in data.txt -out data.enc -K as34Dw9P2IsH openssl enc
i coś tam
```

Z opisu: „tryb wiązania zaszyfrowanych bloków” = CBC (Cipher Block Chaining), a „bloki 256 bitów” w tym kontekście oznacza AES-256 (długość klucza 256 bit, bo AES ma zawsze blok 128-bitowy).

Czyli poprawna komenda (szyfrowanie, nie deszyfrowanie) to ta z:

- enc -e (encrypt)
- -aes-256-cbc
- -in data.txt -out data.enc
- hasło podane jako -k as34Dw9P2IsH (hasło), a nie -K (hex-klucz)

✓ openssl enc -e -aes-256-cbc -in data.txt -out data.enc -k as34Dw9P2IsH

Dlaczego pozostałe widoczne odpadają

- ... -aes-256-ecb ... ❌ bo ECB to nie CBC
- ... -d -aes-256-cbc ... ❌ bo -d to deszyfrowanie
- ... -K as34... ❌ bo -K oczekuje klucza w hex, a tu jest hasło (tekst)

Jeśli brakująca odpowiedź była zastąpiona, to i tak szukaj tej, która dokładnie odpowiada wzorcowi powyżej: -e -aes-256-cbc ... -k <hasło>.

W kryptografii asymetrycznej: *

- ☐ wysyłane wiadomości szyfruje się kluczami publicznymi nadawcy
- ☐ wysyłane wiadomości szyfruje się kluczami prywatnymi nadawcy
- ☒ wysyłane wiadomości szyfruje się kluczami publicznymi odbiorcy
- ☐ wysyłane wiadomości szyfruje się kluczami prywatnymi odbiorcy

W kryptografii asymetrycznej:

wysyłane wiadomości szyfruje się kluczami publicznymi nadawcy

wysyłane wiadomości szyfruje się kluczami prywatnymi nadawcy

☒ wysyłane wiadomości szyfruje się kluczami publicznymi odbiorcy

wysyłane wiadomości szyfruje się kluczami prywatnymi odbiorcy

Poprawna odpowiedź (dla szyfrowania „żeby tylko odbiorca mógł przeczytać”) to:

☒ wysyłane wiadomości szyfruje się kluczami publicznymi odbiorcy

Wyjaśnienie

W kryptografii asymetrycznej do poufności działa to tak:

- Nadawca bierze klucz publiczny odbiorcy i nim szyfruje wiadomość.
- Odbiorca odszyfrowuje ją swoim kluczem prywatnym.

Pozostałe opcje są błędne w kontekście poufności:

- ❌ publiczny nadawcy – każdy go zna, więc nie zapewnia poufności dla odbiorcy
- ❌ prywatny nadawcy – to byłby raczej mechanizm podpisu (w uproszczeniu), nie standardowe szyfrowanie do poufności
- ❌ prywatny odbiorcy – prywatnym kluczem się nie „wysyła” do szyfrowania; on jest do odszyfrowania i powinien pozostać tajny

Które elementy kodu mogą prowadzić do podatności XSS? *

```
echo "<div>Welcome " . $_GET['name'] . "</div>";  
echo "<script>var data = " . $_POST['data'] . "</script>";  
document.write(location.hash.substring(1));  
$('div').html(userInput);
```

☒ Bezpośrednie użycie `$_GET['name']` w HTML

☒ Wstawianie `$_POST['data']` do JavaScript

☐ Użycie protokołu HTTP

☐ Brak walidacji długości danych

☒ Używanie jQuery

Zaloguj się, by kontynuować

Które elementy kodu mogą prowadzić do podatności XSS? *

```
echo "<div>Welcome " . $_GET['name'] . "</div>"; echo "<script>var data = " .  
$_POST['data'] . "</script>"; document.write(location.hash.substring(1));  
$('div').html(userinput);
```

☒ Bezpośrednie użycie `$_GET['name']` w HTML

☒ Wstawianie `$_POST['data']` do JavaScript

☐ Użycie protokołu HTTP

☐ Brak walidacji długości danych

☒ Używanie jQuery...

W tym pytaniu „elementy kodu” prowadzące do XSS to dokładnie te miejsca, gdzie dane od użytkownika trafiają do HTML/JS bez escapingu/sanitizacji. Z widocznych checkboxów zaznacz:

☒ Bezpośrednie użycie `$_GET['name']` w HTML

☒ Wstawianie `$_POST['data']` do JavaScript

☒ (ta ostatnia ucięta) Używanie jQuery `.html(userInput)` (to praktycznie jak `innerHTML`, bardzo częsty wektor XSS)

I nie zaznaczaj:

✗ Użycie protokołu HTTP (to nie jest przyczyna XSS)

✗ Brak walidacji długości danych (długość nie rozwiązuje XSS; potrzebny jest escape/sanitizacja)

Dodatkowo: w samym fragmencie kodu jest jeszcze

`document.write(location.hash.substring(1));` — jeśli wśród odpowiedzi masz opcję odnoszącą się do `document.write/location.hash`, to też ją zaznacz (to również typowy DOM XSS).

Na podstawie analizy zawartości katalogu fd dla pewnego procesu możemy stwierdzić, że:

```
dr-x----- 2 kali kali 0 Jan 6 16:37 .
dr-xr-xr-x 9 kali kali 0 Jan 6 16:37 ..
lr-x----- 1 kali kali 64 Jan 6 16:37 0 -> /dev/null
l-wx----- 1 kali kali 64 Jan 6 16:37 1 -> /home/kali/.xsession-errors
lrwx----- 1 kali kali 64 Jan 6 16:37 10 -> 'socket:[20450]'
l-wx----- 1 kali kali 64 Jan 6 16:37 2 -> /home/kali/.xsession-errors
lrwx----- 1 kali kali 64 Jan 6 16:37 3 -> 'socket:[13283]'
lrwx----- 1 kali kali 64 Jan 6 16:37 4 -> 'anon_inode:[eventfd]'
lrwx----- 1 kali kali 64 Jan 6 16:37 5 -> 'socket:[13285]'
lrwx----- 1 kali kali 64 Jan 6 16:37 6 -> 'anon_inode:[eventfd]'
lrwx----- 1 kali kali 64 Jan 6 16:37 7 -> 'anon_inode:[eventfd]'
lrwx----- 1 kali kali 64 Jan 6 16:37 8 -> 'socket:[22601]'
lrwx----- 1 kali kali 64 Jan 6 16:37 9 -> 'anon_inode:[eventfd]'
```

- ☒ informacje diagnostyczne zapisywane są w pliku `.xsession-errors`
- ☒ nie ma możliwości przekazania do procesu nic ze standardowego wejścia
- ☐ proces zapisuje informacje o błędach do pliku `socket:[13283]`
- ☐ proces do komunikacji używa plików `/dev/null`

Na podstawie analizy zawartości katalogu fd dla pewnego procesu możemy stwierdzić, że:

```

dr-x 2 kali kali 8 Jan dr-xr-xr-x 9 kali kali 9 Jan lr-x----- 1 kali kali 64 Jan 1-
wx----- 1 kali kali 64 Jan 1 kali kali 64 Jan l-wx-- 1 kali kali 64 Jan Lrwx-.. 1
kali kali 64 Jan 1 kali kali 64 Jan Lrwx----- 1 kali kali 64 Jan lrwx--- 1 kali
kali 64 Jan 1 kali kali 64 Jan lrwx 1 kali kali 64 Jan
6 16:37
6 16:37
6 16:37 0
/dev/null
->
socket: 20450
6 16:37 1-> /home/kali/.xsession-errors 6 16:37 6 16:37 2> /home/kali/.xsession-
errors 6 16:37 socket (132831 6 16:37
6 16:37 6 16:37
6 16:37
6 16:37
1 kali kali 64 Jan 6 16:37
ance lode [eventfd socket 13285)
Anop snode: leventfd and inode [eventfd socket 226011
and mode leventfd

```

- ✓ informacje diagnostyczne zapisywane są w pliku xsession-errors
 - ✓ nie ma możliwości przekazania do procesu nic ze standardowego wejścia
- proces zapisuje informacje o błędach do pliku socket:[13283]
- proces do komunikacji używa plików /dev/null

Z tego listingu /proc//fd widać:

- 0 → /dev/null (stdin)
- 1 → /home/kali/.xsession-errors (stdout)
- 2 → /home/kali/.xsession-errors (stderr)
- dalej są różne socket:[...] i anon_inode:[eventfd]

Dlatego poprawne są dwie pierwsze odpowiedzi:

- ✓ „informacje diagnostyczne zapisywane są w pliku xsession-errors”

— bo zarówno stdout (1) jak i stderr (2) idą do ~/.xsession-errors.

- ✓ „nie ma możliwości przekazania do procesu nic ze standardowego wejścia”

— bo stdin (0) jest podpięty do /dev/null, więc proces nie dostanie realnego wejścia z klawiatury/potoku (odczyt to praktycznie od razu EOF).

A te dwie są niepoprawne:

- ✗ „proces zapisuje informacje o błędach do pliku socket:[13283]”

— socket:[13283] to po prostu gniazdo na fd 3 (komunikacja), ale błędy lecą na fd 2 do .xsession-errors.

✗ „proces do komunikacji używa plików /dev/null”

— /dev/null jest użyte jako stdin, nie jako kanał komunikacji.

Jeden z parametrów połączenia HTTPS ma wartość TLS_ECDHE_ECDSA_WITH_AES256_GCM_SHA384. Oznacza to, że:

- ☒ do szyfrowania szyfrowania wykorzystywany jest algorytm Diffie-Hellman
- ☐ do podpisywania wiadomości wykorzystuje się algorytm AES256
- ☐ do wygenerowania MAC wykorzystywana jest funkcja SHA384
- ☐ do szyfrowania wiadomości wykorzystuje się algorytm DSA
- ☐ wymienione przez obie strony kucze należą do cryptosystemu RSA

Jeden z parametrów połączenia HTTPS ma wartość TLS_ECDHE_ECDSA_WITH_AES256_GCM_SHA384. Oznacza to, że:

- ☒ do szyfrowania szyfrowania wykorzystywany jest algorytm Diffie-Hellman
- do podpisywania wiadomości wykorzystuje się algorytm AES256
- do wygenerowania MAC wykorzystywana jest funkcja SHA384
- do szyfrowania wiadomości wykorzystuje się algorytm DSA
- wymienione przez obie strony kucze należą do cryptosystemu RSA

TLS_ECDHE_ECDSA_WITH_AES256_GCM_SHA384 rozkłada się tak:

- ECDHE – uzgadnianie klucza (wymiana) metodą ephemeral Elliptic-Curve Diffie-Hellman
- ECDSA – uwierzytelnianie/podpis (certyfikat serwera i podpisy w handshake)
- AES256_GCM – szyfrowanie danych + integralność w trybie AEAD (GCM daje „MAC/tag” wbudowany)
- SHA384 – hash używany w TLS 1.2 głównie w PRF/handshake, a nie jako osobny MAC dla danych aplikacyjnych (bo to robi GCM)

Zaznacz:

☒ „do szyfrowania ... wykorzystywany jest algorytm Diffie-Hellman”

– sensownie chodzi o wymianę/uzgadnianie klucza (ECDHE to odmiana Diffie-Hellmana na krzywych eliptycznych).

Nie zaznaczaj:

✗ „do podpisywania wiadomości wykorzystuje się AES256” (AES to szyfr symetryczny, nie podpis)

✗ „do wygenerowania MAC wykorzystywana jest SHA384” (przy AES-GCM integralność zapewnia GCM, SHA384 nie jest MAC-em danych)

✗ „do szyfrowania wiadomości wykorzystuje się DSA” (jest ECDSA, nie DSA do szyfrowania)

✗ „wymienione klucze należą do RSA” (tu jest EC: ECDHE/ECDSA)

Wykorzystanie algorytmu PBKDF2 podczas szyfrowania pozwala na: *

- ☐ zmniejszenie rozmiaru pliku po zaszyfrowaniu
- ☐ wykorzystanie silniejszego algorytmu szyfrującego
- ☐ zabezpieczenie przed atakiem mającym na celu złamanie algorytmu szyfrującego
- ☒ zwiększeniu kosztów obliczeniowych w celu redukcji podatności na atak typu "brut force"
- ☐ redukcje kosztów obliczeniowych związanych z wyodrębnieniem klucza szyfrującego z hasła

Wykorzystanie algorytmu PBKDF2 podczas szyfrowania pozwala na:

zmniejszenie rozmiaru pliku po zaszyfrowaniu
wykorzystanie silniejszego algorytmu szyfrującego
zabezpieczenie przed atakiem mającym na celu złamanie algorytmu szyfrującego
☒ zwiększeniu kosztów obliczeniowych w celu redukcji podatności na atak typu "brut force"
redukcje kosztów obliczeniowych związanych z wyodrębnieniem klucza szyfrującego z hasła

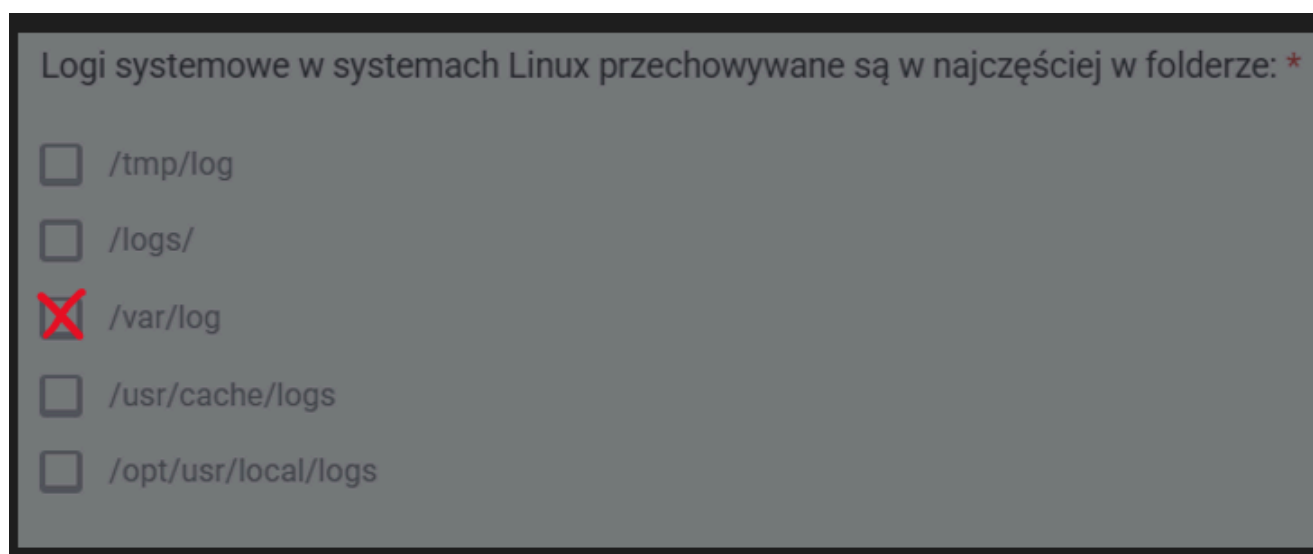
PBKDF2 to funkcja wyprowadzania klucza z hasła (Key Derivation Function), która robi to celowo wolniej przez wiele iteracji, żeby utrudnić zgadywanie haseł.

Dlatego poprawna odpowiedź to:

✓ „zwiększeniu kosztów obliczeniowych w celu redukcji podatności na atak typu 'brut force' ”

Pozostałe są błędne:

- ✗ zmniejszenie rozmiaru pliku po zaszyfrowaniu — szyfrowanie zwykle nie kompresuje, często nawet dodaje narzut
- ✗ wykorzystanie silniejszego algorytmu szyfrującego — PBKDF2 nie zmienia szyfru (np. AES), tylko sposób uzyskania klucza z hasła
- ✗ zabezpieczenie przed atakiem „złamaniem algorytmu szyfrującego” — KDF nie „naprawia” słabości szyfru
- ✗ redukcja kosztów obliczeniowych wyodrębnienia klucza z hasła — jest odwrotnie: podnosi koszty celowo



Logi systemowe w systemach Linux przechowywane są w najczęściej w folderze: *

/tmp/log

/logs/

✓ /var/log

/usr/cache/logs

/opt/usr/local/logs

Poprawna odpowiedź to:

✓ /var/log

W większości dystrybucji Linuksa standardowe logi systemowe i usług (np. syslog, messages, auth.log, kern.log, journal-related pliki, logi Apache/Nginx itd.) trzyma się właśnie w /var/log.

Pozostałe opcje typu /tmp/log, /logs/, /usr/cache/logs, /opt/usr/local/logs nie są standardowymi lokalizacjami logów systemowych.

Wykonanie polecenia "echo 'password' | shasum -a 256" spowoduje wygenerowanie ciągu:

```
☐ 398f71dGdfecdadbfa5f638d87e40a6e604e1f8697c4e2bafBbcf772daff==  
☒ 398f71d9dfecdadbfa5f638d87e40a6e604e1f8697c4e2baf8bcf772daff8b  
☐ 398f71d9dfe7e25a17a322dcc80d77a23d40ebd20ec  
☐ 0000000 39 8f 71 d9 6f 72 64 0a 3a 23 45 a1 ec 20
```

Wykonanie polecenia "echo 'password' | shasum -a 256" spowoduje wygenerowanie ciągu:

```
398f71dGdfecdadbfa5f638d87e40a6e604e1f8697c4e2bafBbcf772daff==  
☒ 398f71d9dfecdadbfa5f638d87e40a6e604e1f8697c4e2baf8bcf772daff8b  
398f71d9dfe7e25a17a322dcc80d77a23d40ebd20ec  
0000000 39 8f 71 d9 6f 72 64 0a 3a 23 45 a1 ec 20
```

Polecenie:

```
echo 'password' | shasum -a 256
```

zrobi SHA-256 z tekstu password + znaku nowej linii (\n), bo zwykle echo dopisuje newline na końcu.

Dlatego wygenerowany skrót (hex, 64 znaki) to:

☒ 398f71d9dfecdadbfa5f638d87e40a6e604e1f8697c4e2baf8bcf772daff8b (to jest 2. odpowiedź na screenie)

Dla porównania: gdyby było echo -n 'password' | shasum -a 256, hash byłby inny (bez \n).

Bo SHA-256 ma wynik o długości 256 bitów, a na screenie wynik jest wypisany w hexie.

- 1 znak hex = 4 bity (bo hex to zapis w systemie 16: 0–f).
- 256 bitów ÷ 4 bity/znak = 64 znaki.

Można to też policzyć bajtami:

- 256 bitów = 32 bajty,

- każdy bajt to 2 znaki hex,
- $32 \times 2 = 64$ znaki.

Aby zaszyfrować plik data.txt, o rozmiarze 4 kB wykorzystując klucz publiczny o rozmiarze 4 kb (zapisany w pliku pub.pem) należy:

- ☐ wykonać polecenie: `openssl rsautl -encrypt -inkey pub.pem -pubin -in data.txt -out data.enc`
- ☐ wykonać polecenie: `openssl rsautl -encrypt -inkey priv.pem -in data.txt -out data.enc`
- ☒ wygenerować losowy klucz, zaszyfrować podany plik za pomocą algorytmu symetrycznego z wykorzystaniem wygenerowanego klucza, zaszyfrować kluczem publicznym klucz symetryczny, ewentualnie przesłać odbiorcy zaszyfrowany plik wraz z zaszyfrowanym kluczem
- ☐ podpisać wskazany plik za pomocą klucza publicznego, zaszyfrować podpisany plik za pomocą algorytmu symetrycznego z wykorzystaniem klucza prywatnego
- ☐ zaimportować klucz do systemu gpg i wykonać polecenie: `gpg --encrypt --key pub.pem -in data.txt`

Aby zaszyfrować plik data.txt, o rozmiarze 4 kB wykorzystując klucz publiczny o rozmiarze 4 kb (zapisany w pliku pub.pem) należy:

wykonać polecenie: `openssl rsautl -encrypt -inkey pub.pem -pubin -in data.txt -out data.enc`

wykonać polecenie: `openssl rsautl -encrypt -inkey priv.pem -in data.txt -out data.enc`

☒ wygenerować losowy klucz, zaszyfrować podany plik za pomocą algorytmu symetrycznego z wykorzystaniem wygenerowanego klucza, zaszyfrować kluczem publicznym klucz symetryczny, ewentualnie przesłać odbiorcy zaszyfrowany plik wraz z zaszyfrowanym kluczem

podpisać wskazany plik za pomocą klucza publicznego, zaszyfrować podpisany plik za pomocą algorytmu symetrycznego z wykorzystaniem klucza prywatnego

zaimportować klucz do systemu gpg i wykonać polecenie: `gpg --encrypt --key pub.pem --in data.txt`

W tym pytaniu poprawna jest ta odpowiedź o "hybrid encryption" (czyli: losowy klucz symetryczny + szyfrowanie pliku symetrycznie + zaszyfrowanie klucza symetrycznego kluczem publicznym).

✓ „wygenerować losowy klucz... zaszyfrować plik algorytmem symetrycznym... zaszyfrować klucz symetryczny kluczem publicznym...”

Dlaczego tak?

RSA (nawet z kluczem 4096 bit) nie służy do szyfrowania dużych plików bezpośrednio:

- RSA ma ograniczenie rozmiaru danych wejściowych (można zaszyfrować tylko wiadomość krótszą od modułu, i jeszcze mniej przez padding).
- Do plików używa się więc standardowo podejścia hybrydowego: szybki szyfr symetryczny do danych + RSA tylko do klucza.

Dlaczego pozostałe odpadają

- ✗ `openssl rsautl -encrypt ... -pubin -in data.txt` — nawet jeśli polecenie jest “logicznie” o publicznym kluczu, nie zaszyfruje 4 kB bezpośrednio RSA (za duże).
- ✗ `rsautl z priv.pem` — szyfrowanie „prywatnym” to nie jest standardowa poufność (to myli się z podpisem), a poza tym rozmiar dalej problem.
- ✗ „podpisać kluczem publicznym...” — podpis robi się prywatnym, weryfikuje publicznym, więc opis jest błędny.
- ⚠ `gpg --encrypt ...` — GPG też używa hybrydowego podejścia, ale ta opcja na screenie wygląda na źle sformułowaną (w praktyce podaje się odbiorcę `-r`, a nie sam plik PEM jako `--key pub.pem`). W testach zwykle chcą odpowiedź koncepcyjną o hybrydowym szyfrowaniu.

Jeśli to pytanie wymaga jednej odpowiedzi — wybierz `hybrid encryption` (ta zaznaczona u Ciebie).

W przedstawionej na poniższym rysunku sytuacji mamy do czynienia z:

```
(kali@kali)-[/bin]
$ php8.1 /root/.bashrc
# ~/.bashrc: executed by bash(1) for non-login shells.
# see /usr/share/doc/bash/examples/startup-files (in the package bash-doc)
# for examples

# If not running interactively, don't do anything
case $- in
    *i*) ;;
    *) return;;
esac
```

- ☒ eskalacją uprawnień użytkownika
- ☐ wykorzystaniem polecenia "sudo"
- ☒ błędami w nadawaniu uprawnień programom
- ☐ nieudaną próbą dostępu do plików w katalogu domowym użytkownika root
- ☐ zmianą praw dostępu do pliku .bashrc

W przedstawionej na poniższym rysunku sytuacji mamy do czynienia z:

- ☒ eskalacją uprawnień użytkownika
- wykorzystaniem polecenia „sudo”
- ☒ błędami w nadawaniu uprawnień programom
- nieudaną próbą dostępu do plików w katalogu domowym użytkownika root
- zmianą praw dostępu do pliku ``.bashrc``

Na tym screenie widać, że jako użytkownik kali uruchamiasz:

`php8.1 /root/.bashrc`

i udaje się odczytać zawartość pliku z katalogu /root (widać linie z .bashrc). Normalnie użytkownik nie-root nie ma dostępu do /root, więc to oznacza, że proces php8.1 działa z podwyższonymi uprawnieniami (np. ma ustawiony SUID/SGID albo jest źle skonfigurowany w systemie).

Dlatego poprawnie pasują:

- ☒ eskalacją uprawnień użytkownika
- ☒ błędami w nadawaniu uprawnień programom

A te nie pasują:

✗ wykorzystaniem polecenia sudo (tu go nie ma)

✗ nieudaną próbą dostępu do plików w katalogu domowym roota (bo dostęp się udał)

✗ zmianą praw dostępu do pliku .bashrc (na screenie nie ma chmod/chown, tylko odczyt)

Jeśli to test jednokrotnego wyboru, najbliższa „jedna” odpowiedź to zwykle: błędy w nadawaniu uprawnień programom (bo to przyczyna), ale merytorycznie sytuacja pokazuje też eskalację uprawnień (skutek).